Status	Finished
Started	Sunday, 11 October 2026, 2:01 PM
Completed	Sunday, 11 October 2026, 2:11 PM
Duration	9 mins 49 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import re
from collections import Counter

def count_word_frequencies():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Convert text to lowercase and remove punctuation markers
    cleaned_text = re.sub(r'[^\w\s]', '', input_data.lower())
    words = cleaned_text.split()
    counts = Counter(words)

    with open('output.txt', 'w') as file:
        for word in sorted(counts.keys()):
            file.write(f"{word}: {counts[word]}\n")

if __name__ == "__main__":
    count_word_frequencies()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def reverse_specific_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    with open(input_filename, 'r') as file:
        file_lines = file.read().splitlines()

    if 1 <= line_number <= len(file_lines):
        target_idx = line_number - 1
        line = file_lines[target_idx]

        # Check if line ends with a period
        if line.endswith('.'):
            reversed_line = line[:-1][::-1] + '.'
        else:
            reversed_line = line[::-1]

        file_lines[target_idx] = reversed_line

```

```
    with open('output.txt', 'w') as file:
        for line in file_lines:
            file.write(line + '\n')

if __name__ == "__main__":
    reverse_specific_line()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def count_words_in_file():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    input_filename = input_data

    try:
        with open(input_filename, 'r') as file:
            content = file.read()
            words = content.split()
            print(f"Total words: {len(words)}")
    except FileNotFoundError:
        print("Total words: 0")

if __name__ == "__main__":
    count_words_in_file()

```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys

def find_longest_word():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    input_filename = input_data

    try:
        with open(input_filename, 'r') as file:
            content = file.read()

            words = content.split()

            if not words:
                print("Longest word: ")
                return

            longest_word = ""
            for word in words:
                cleaned_word = "".join(char for char in word if char.isalnum() or char in "-'")

                if len(word) > len(longest_word):
                    longest_word = word

            print(f"Longest word: {longest_word}")
    except FileNotFoundError:
        print("Longest word: ")

if __name__ == "__main__":
    find_longest_word()

```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import sys

def find_file_palindromes():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    input_filename = input_data

    try:
        with open(input_filename, 'r') as file:
            content = file.read()

        words = content.split()
        palindrome_words = []

        for word in words:

            cleaned_word = "".join(char for char in word if char.isalnum())
            if not cleaned_word:
                continue


            if cleaned_word.lower() == cleaned_word.lower()[::-1]:
                palindrome_words.append(word)

        with open('output.txt', 'w') as out_file:
            for word in palindrome_words:
                out_file.write(word + '\n')

    except FileNotFoundError:
        with open('output.txt', 'w') as out_file:
            pass

if __name__ == "__main__":
    find_file_palindromes()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`	`madam`
			`arora`
			`malayalam`
				`madam`
				`arora`
				`malayalam`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def insert_line_at_position():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    with open(input_filename, 'r') as file:
        file_lines = file.read().splitlines()

    target_idx = line_number - 1

    if target_idx < 0:
        target_idx = 0

    if target_idx >= len(file_lines):
```

```
        file_lines.append(new_content)
    else:
        file_lines.insert(target_idx, new_content)

    with open('output.txt', 'w') as file:
        for line in file_lines:
            file.write(line + '\n')

if __name__ == "__main__":
    insert_line_at_position()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def delete_specific_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    with open(input_filename, 'r') as file:
        file_lines = file.read().splitlines()

    if 1 <= line_number <= len(file_lines):
        file_lines.pop(line_number - 1)

    with open('output.txt', 'w') as file:
        for line in file_lines:
            file.write(line + '\n')

if __name__ == "__main__":
    delete_specific_line()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	output1.txt
		This is the source file.
		It contains multiple lines of text.
		Here is another line.

Answer: (penalty regime: 0 %)

```
import sys

def copy_file_contents():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    source_filename = input_data[0].strip()
    destination_filename = input_data[1].strip()

    try:
        with open(source_filename, 'r') as source_file:
            content = source_file.read()

        with open(destination_filename, 'w') as dest_file:
            dest_file.write(content)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    copy_file_contents()

```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		output1.txt
			This is the source file.
			It contains multiple lines of text.
			Here is another line.
				This is the source file.
				It contains multiple lines of text.
				Here is another line.
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		output2.txt
			Hello, world!
			Python programming is amazing.
			Let's copy this text to another file.
				Hello, world!
				Python programming is amazing.
				Let's copy this text to another file.
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input3.txt
		output3.txt
			Single line.	Single line.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def read_specific_line():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    try:
        with open(input_filename, 'r') as file:
            file_lines = file.read().splitlines()

        if 1 <= line_number <= len(file_lines):
            print(file_lines[line_number - 1])
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    read_specific_line()
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def replace_specific_line():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 3:
        return

    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    try:
        with open(input_filename, 'r') as file:
            file_lines = file.read().splitlines()
    except FileNotFoundError:
        return

    if 1 <= line_number <= len(file_lines):
        file_lines[line_number - 1] = new_content

    with open('output.txt', 'w') as file:
        for line in file_lines:
```

```
            file.write(line + '\n')

if __name__ == "__main__":
    replace_specific_line()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.